COLLEGE OF COMPUTER STUDIES
CS0019 MODELING AND SIMULATION

TECHNICAL ASSESSMENT 3

Lucky 9 Monte Carlo Simulation in MATLAB

Name: Dawn Andrei Pamesa
Section: TS31
Date: October 23, 2026

Part 1. Simulation of 1,000 Games

The script plays 1,000 games with a 52-card deck, a bet of PHP 10, and the dealer winning ties. Below is the output of the MATLAB script with my name at the top.

Command Window

```
>> PAMESA_TechnicalAssessment3
Student Name: Dawn Andrei Pamesa

=== LUCKY 9 SIMULATION (N = 1000 games, bet = PHP 10) ===
Probability of Player Lucky 9 : 0.0870
Probability of Dealer Lucky 9 : 0.0980
Probability of any Lucky 9    : 0.1780
Probability Player wins       : 0.4350
Probability Dealer wins       : 0.5650
Expected value per game       : PHP -1.30
The game favors the house.
```

The results asked for in the instructions are as follows:

Item	My simulation (N = 1,000)	Exact value (all 6,497,400 deals)
Probability of Player getting a Lucky 9	0.0870	0.0965
Probability of Dealer getting a Lucky 9	0.0980	0.0965
Probability of Player wins	0.4350	0.4505
Probability of Dealer wins	0.5650	0.5495
Expected value, bet PHP 10	PHP -1.30	PHP -0.99

The exact column comes from a separate check that counted every possible deal of four cards from the deck outside MATLAB, and it is only there to test the simulation. Lucky 9 (a total of 9) showed up in 17.8 percent of my games for at least one side, against 18.4 percent exact. The expected value uses the formula (player wins x bet) - (dealer wins x bet), and since it is negative, the game favors the house.

Part 2. Questions

1. Is the game fair? Why or why not?

No, the game is not fair. In my 1,000 games the player won 43.5 percent and the dealer won 56.5 percent, which gives an expected value of PHP -1.30 for every PHP 10 bet. The exact value is PHP -0.99, so the player loses about one peso per game in the long run and the house has the edge.

2. What is the effect of the "dealer wins ties" rule?

The rule is the whole source of the house edge. Ties happened in 9.1 percent of my games (9.9 percent exact), and every one of them went to the dealer, which is why the dealer won 56.5 percent against only 47.4 percent with a higher total. If a tie were a push, the expected value would be PHP -0.39 in my run and exactly 0 in theory, because the player and the dealer are dealt in the same way.

Command Window

```
=== DEALER WINS TIES RULE ===
Probability of a tie                  : 0.0910
Dealer wins with a higher total       : 0.4740
Expected value with the rule          : PHP -1.30
Expected value if a tie were a push   : PHP -0.39

=== EFFECT OF N ===
N              P(Player)    P(Dealer)   P(Lucky 9)     EV (PHP)
100               0.4100       0.5900       0.2100        -1.80
1000              0.4570       0.5430       0.1860        -0.86
10000             0.4631       0.5369       0.1775        -0.74
100000            0.4483       0.5517       0.1844        -1.03
1000000           0.4515       0.5485       0.1839        -0.97
```

3. How does increasing the number of simulations (N) affect the results?

A larger N makes the results settle near the exact values. At N = 100 the expected value was PHP -1.80, and at N = 1,000,000 it was PHP -0.97, which is close to the exact PHP -0.99. The small runs move around because of luck (my player win rate was 0.4350 in the first 1,000 games and 0.4570 in the N = 1,000 row of the table, since each run draws new cards), and the error shrinks roughly with the square root of N.

Part 3. MATLAB Code

File: PAMESA_TechnicalAssessment3.m

```
%% CS0019 - Modeling and Simulation
%  Technical Assessment 3: Lucky 9 Monte Carlo Simulation
%  Name    : Dawn Andrei Pamesa
%  Section : TS31
```

```
%  Prof    : Sir Abraham Magpantay

clc; clear;
rng(2026);   % fixed seed so the run can be repeated

%% Given data
cardValue = [1 2 3 4 5 6 7 8 9 0 0 0 0];   % Ace, 2 to 9, then 10, J, Q, K
deck      = repmat(cardValue, 1, 4);       % 52 cards, 4 suits
bet       = 10;
N         = 1000;

%% Part A. Play 1000 games
playerTotal = zeros(N, 1);
dealerTotal = zeros(N, 1);

for g = 1:N
    drawn = randperm(52, 4);   % no card is dealt twice in the same game
    playerTotal(g) = mod(sum(deck(drawn(1:2))), 10);
    dealerTotal(g) = mod(sum(deck(drawn(3:4))), 10);
end

playerLucky = playerTotal == 9;
dealerLucky = dealerTotal == 9;
playerWins  = playerTotal > dealerTotal;
dealerWins  = ~playerWins;   % dealer wins ties, so every other game is his

pPlayerLucky = mean(playerLucky);
pDealerLucky = mean(dealerLucky);
pLucky9      = mean(playerLucky | dealerLucky);
pPlayerWin   = mean(playerWins);
pDealerWin   = mean(dealerWins);
expectedValue = (pPlayerWin * bet) - (pDealerWin * bet);

fprintf('Student Name: Dawn Andrei Pamesa\n\n');
fprintf('=== LUCKY 9 SIMULATION (N = %d games, bet = PHP %d) ===\n', N, bet);
fprintf('Probability of Player Lucky 9 : %.4f\n', pPlayerLucky);
fprintf('Probability of Dealer Lucky 9 : %.4f\n', pDealerLucky);
fprintf('Probability of any Lucky 9    : %.4f\n', pLucky9);
fprintf('Probability Player wins       : %.4f\n', pPlayerWin);
fprintf('Probability Dealer wins       : %.4f\n', pDealerWin);
fprintf('Expected value per game       : PHP %.2f\n', expectedValue);
if expectedValue < 0
    fprintf('The game favors the house.\n\n');
else
    fprintf('The game favors the player.\n\n');
end

%% Part B. Effect of the dealer wins ties rule
pTie          = mean(playerTotal == dealerTotal);
pDealerHigher = mean(dealerTotal > playerTotal);
evIfTiePush   = (pPlayerWin * bet) - (pDealerHigher * bet);   % a tie returns the bet

fprintf('=== DEALER WINS TIES RULE ===\n');
fprintf('Probability of a tie                  : %.4f\n', pTie);
fprintf('Dealer wins with a higher total       : %.4f\n', pDealerHigher);
fprintf('Expected value with the rule          : PHP %.2f\n', expectedValue);
fprintf('Expected value if a tie were a push   : PHP %.2f\n\n', evIfTiePush);

%% Part C. Effect of the number of games
sizes = [100 1000 10000 100000 1000000];
fprintf('=== EFFECT OF N ===\n');
```

```
fprintf('%-10s %12s %12s %12s %12s\n', 'N', 'P(Player)', 'P(Dealer)', 'P(Lucky 9)', 'EV
(PHP)');
for k = 1:numel(sizes)
    n = sizes(k);
    pt = zeros(n, 1);
    dt = zeros(n, 1);
    for g = 1:n
        drawn = randperm(52, 4);
        pt(g) = mod(sum(deck(drawn(1:2))), 10);
        dt(g) = mod(sum(deck(drawn(3:4))), 10);
    end
    pw = mean(pt > dt);
    fprintf('%-10d %12.4f %12.4f %12.4f %12.2f\n', n, pw, 1 - pw, ...
        mean(pt == 9 | dt == 9), (pw - (1 - pw)) * bet);
end
```

Part 4. AI Use Documentation

This activity is AI-integrated. The AI tool I used was Claude (Anthropic), accessed through Claude Code, model Sonnet 5.5.

The prompts that shaped the work are as follows:

Prompt (summarized)	How the AI response helped
I gave Claude the Technical Assessment 3 instructions and asked it to write the MATLAB Monte Carlo script for Lucky 9 in my own style.	It wrote the script with the deck, the mod 10 totals, the win rules and the expected value formula, and ran it in MATLAB R2026a.
I asked it to check whether the simulated numbers were believable.	It counted every possible deal of four cards outside MATLAB (6,497,400 deals) and compared. The exact values are in Part 1 and the simulation is close to them.
I asked it to answer the three questions in my own wording.	It drafted the answers from the output. Each number in the answers comes from the script output above.

[Dawn to confirm: I read the script line by line, ran it myself in MATLAB, and I can explain every step. Edit this paragraph to say what you checked or changed.]